

A Deadlock-free Design: Bubble Flow Control on 2D-Torus

Student A
IIS
Tsinghua University
BeiJing, China
A@mails.tsinghua.edu.cn

Student B
IIS
Tsinghua University
Beijing, China
B@mails.tsinghua.edu.cn

I. INTRODUCTION

Recently, there has been more and more discussion about NoC, since Moore's Law has failed on a single CPU core. The topology chosen by the NoC is crucial, directly determining the performance parameters of the network, such as **Diameter**, **Degree**, **Bisection Bandwidth**, **Avg Distance**. The most common topology is the 2D mesh, due to its ease of implementation and ability to maintain a certain level of performance. However, all the performance parameters of 2D-torus could surpass that of 2D-mesh, which only requires the addition of a small number of wraparound links. People are not happy with 2D-torus because the toroidal architecture is not only difficult to implement in chips, but can also lead to deadlocks.

II. OUR WORK

To put 2D-torus into practice, we implement three types of bubble flow control - including a simple scheme, a critical bubble scheme and a multiple critical bubble scheme. These schemes added to 2D-torus can improve throughput and reduce packet latency compared to 2D-mesh, while avoiding deadlocks. We run experiments on different synthetic types and we also discuss the pros and cons of the three schemes that can be used in different situations.

III. 2D-TORUS & BUBBLE FLOW CONTROL

A. 2D-Torus

The performance parameters of a $k \times k$ 2D-Torus:

- Diameter:
$$\frac{k}{2}(\text{x-axis}) + \frac{k}{2}(\text{y-axis}) = k$$
- Degree: 4
- Bisection Bandwidth: $2k$
- Avg Distance: $\frac{k^2}{4(k-1)}$

For Ring of length k , average distance is:

$$\frac{1}{k(k-1)}k(2(1+2+\dots+\frac{k}{2}-1)+\frac{k}{2}) = \frac{k^2}{4(k-1)}$$

Thus for $k \times k$ 2D-Torus, Avg distance is $\frac{k^2}{2(k-1)}$.

In contrast with 2D-Mesh:

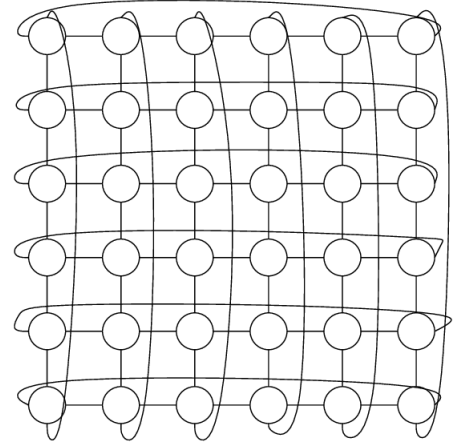


Fig. 1. 2D-Torus

TABLE I
2D-TORUS VS 2D-MESH

	2D-Torus	2D-Mesh
Diameter	k	$2k$
Degree	4	4
Bisection Bandwidth	$2k$	k
Avg Distance	$\frac{k^2}{2(k-1)}$	$\frac{2k}{3}$

It is easy to see that 2D-Torus has higher bandwidth and lower average distance, which means it can increase throughput while reducing latency. However, it can easily lead to deadlock as in Ring.

B. Bubble Flow Control

When packet load is high, common flow-control mechanisms will fail on simple topologies due to dead-lock, like wormhole on ring or mesh, let alone other simpler flow-control protocols. To address this issue, several improvements have been made. One of them is Bubble Flow control. The main idea of bubble flow control is to leave some unused buffer slots on the network, so that packets do not become so congested as to cause a deadlock. There are many types of bubble flow control mechanisms, varying in the number of bubbles and the method of maintaining the bubbles. In this project we will focus on two of them, **simple bubble** and **critical bubble**.

However, whatever which kind of bubble flow control, as long as it fits the one necessary and sufficient condition, can avoid deadlocks.

Definition 1. Let Q be the set of input queues associated with router nodes. Each queue $q_i \in Q$ has capacity of $cap(q_i)$ measured in the number of flits, and the current number of flits occupying the queue is denoted as $size(q_i)$. For any packet p .

Lemma 1. The deadlock will occur if and only if there is no bubble, i.e.

$$\forall q_i \in Q, cap(q_i) = size(q_i)$$

Proof 1.

- Consider if there is some bubble that's not occupied by flits. Then the flit before the bubble can move, thus there is no deadlock.
- Reversely, if all the bubble are full, no flit can move.

Note that Lemma 1 give us insight, we only need to keep there is a bubble in the network so that deadlock will not occur. And there are two kinds of movement in the network, in-transit and change-dimension. In-transit movement are the flits' movement within a ring, which does not provide extra bubbles for the network, cannot lead to deadlock. On the other hand, change-dimension is a sort of packet injection to the ring thus may squeeze the only bubble thus lead to deadlock. All the different schema of bubble flow controls struggle at the dealing of change-dimension movements.

1) *Simple Bubble mechanism*: To keep at least a bubble in the network, we just need to estimate whether there is another bubble in the network while I want to inject a bubble into the ring. A straightforward idea is that we only inject the flit into the VC where there are at least two bubbles. At the same time, in-transit movements are not affected, as long as the next buffer has spare bubble. Formally, the rules can be formulated as follows:

- When $x = y$ (in-transit), the flit in q_i can go forward to q_j if $size(q_j) < cap(q_j)$
- When $x \neq y$ (change-dimensions), the flit in q_i can go forward to q_j if $size(q_j) < cap(q_j) - 1$

This must be a deadlock-free way, since after every change-dimension movement, there is at least one bubble.

2) *Critical Bubble mechanism*: The above solution is trivial and only consider the information come from local buffer. Thus there are many flits that cannot be injected though there are buffers with sufficient flits in other VC. When the buffer resources are the bottleneck, this causes the performance to be a little slower. Therefore, to utilize more buffers, a maintained global information is required, which brings us the Critical Bubble mechanism.

At initialization, Critical Bubble mechanism we randomly select a bubble as critical bubble. Critical bubble cannot be placed with flits, thus it can be seen as an indication of "spare bubble", a sort of global information. The maintaining of critical bubble is that we can only seize the critical bubble by in-transit movement, and not by injection. The latter movement can leave a spare bubble to set critical bubble. In contract,

the injection cannot. So, a formal rules can be formulated as follows:

- When $x = y$, the flit in q_i can go forward to q_j if $size(q_j) < cap(q_j)$. Note that if q_j is critical bubble and $size(q_j) = cap(q_j) - 1$, then the critical bubble should be transmit backwards q_i
- When $x \neq y$ (change-dimensions), the flit in q_i can go forward to q_j if $size(q_j) < cap(q_j) - 1$ or $size(q_j) = cap(q_j) - 1 \wedge q_j$ is not critical bubble.

IV. IMPLEMENTATION

A. Simple bubble scheme

We use credit to represent the available buffer in virtual channel. When injecting a flit to a router, first the flit will go to the buffer of "Local" InputUnit, so when deciding whether to send the flit to next router, we need to check if the desired virtual channel has at least 2 slots in buffer.

Moreover, when implementing it on 2d-torus, each ring no matter X-direction or Y-direction should follow the rule above, and when flit change dimension, it should be handled the same as injection, which guarantees each ring operates with its own bubble.

Figure 2 shows part of essential code in SwitchAllocator::arbitrate_outputports() to fullfill functions mentioned above.

```
if (m_router->get_net_ptr()->isWormholeEnabled()) {
    outvc = vc_allocate_wormhole(output, inport, invc);
    if (m_router->get_net_ptr()->isBubbleEnabled() && m_router->get_net_ptr()->getBubbleAlgorithm()==1)
        if (input_unit->get_direction() != "Local" && output_unit->get_direction() != "Local")
        {
            critical = output_unit->get_credit_count(outvc) <= output_unit->num_critical_bubble(outvc);
            if (critical)
                assert(change_XY(input_unit->get_direction(), output_unit->get_direction())==false);
        }
    }else{
        if (outvc == -1) {
            // VC Allocation - select any free VC from output
            outvc = vc_allocate(output, inport, invc);
        }
    }
}
```

Fig. 2. Code

B. Critical Bubble Scheme

We add a state to represent #critical bubble in each VC, which is store in the "OutVcState.hh".

```
82
83     private:
84         int m_id ;
85         Tick m_time;
86         VC_state_type m_vc_state;
87         int m_credit_count;
88         int m_max_credit_count;
89         int m_critical_bubble;
90     };
91
92 } // namespace garnet
```

Fig. 3. add an variable to represent #critical bubble in OutVcState

1) *Initialization of Critical Bubble*: To initialize the critical bubble, we perform this by initializing each "OutputPort" after creating each link. OutputPort will be created with initializing each "OutvcState", so that's the time we can add some critical information to the OutVcState. The initialization schema is as follows, since we find that the too centralized initialization will cause a catastrophic injection jam in the network.

```

65 const int NumCols = m_router->get_net_ptr()->getNumCols();
66
67 if (m_router->get_net_ptr()->isWormholeEnabled() && m_router->get_net_ptr()->isBubbleEnabled() && m_router->get_net_ptr()->getTopology() == "Ring")
68 {
69     if (m_router->get_net_ptr()->getTopology() == "Ring")
70     {
71         if (direction == "Local")
72         {
73             if (m_router->get_id() < m_router->get_net_ptr()->getNumCriticalBubble())
74             {
75                 this->add_critical_bubble(0);
76             }
77         }
78     }
79     else if (m_router->get_net_ptr()->getTopology() == "2D-Torus")
80     {
81         int row = m_router->get_id() / NumCols, column = m_router->get_id() % NumCols;
82         if (direction == "East" || direction == "West")
83         {
84             if ((row - column + NumCols) % NumCols < m_router->get_net_ptr()->getNumCriticalBubble())
85             {
86                 this->add_critical_bubble(0);
87             }
88             if (direction == "North" || direction == "South")
89             {
90                 if ((column - row + NumCols) % NumCols < m_router->get_net_ptr()->getNumCriticalBubble())
91                 {
92                     this->add_critical_bubble(0);
93                 }
94             }
95         }
96     }
97 }

```

Fig. 4. Initialization. We use the crossbar schema in the torus, to distribute the critical bubble uniformly over each ring.

2) *Credit Link*: We also modify the credit system of garnet, to let network transmit critical bubble after flits seize the place of critical bubble. Specifically, we add a boolean variable to represent whether there is a transmission of critical bubble.

```

64 class Credit : public flit
65 {
66 public:
67     Credit();
68     Credit(int vc, bool is_free_signal, Tick curTime);
69     Credit(int vc, bool is_free_signal, Tick curTime, bool is_critical_transfer);
70
71     // Functions used by SerDes
72     flit* serialize(int ser_id, int parts, uint32_t bwidth);
73     flit* deserialize(int des_id, int num_flits, uint32_t bwidth);
74     void print(std::ostream& out) const;
75
76 ~Credit();
77
78 bool is_free_signal() { return m_is_free_signal; }
79 bool is_critical_transfer() { return m_is_critical_transfer; }
80 private:
81     bool m_is_free_signal;
82     bool m_is_critical_transfer;
83 };

```

Fig. 5. credit modification

After each transmission in a ring, we check if the flit will take over a critical bubble, if it's, then the credit link will set this variable to "true".

3) *Injection Criteria*: If there is a non-critical bubble, we can inject the packet.

```

inline bool critical_inject_enable() {
    return (m_credit_count > 0) && (m_credit_count > m_critical_bubble);
}

```

Fig. 6. Injection Criteria

V. EVALUATION

Environmental setting:

- Topology: 8x8, bidirectional Torus
- Routing: XY Routing
- Only one VC channel (inherited from lab3 task2 setting)

A. 2D-Torus Vs 2D-Mesh

Figure 7 shows that torus has lower latency than mesh, due to the shorter average hops.

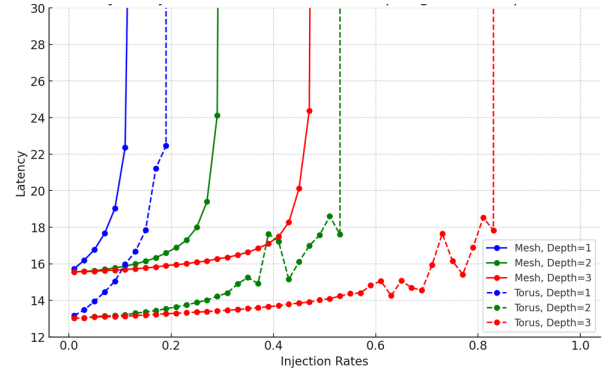


Fig. 7. Mesh VS Torus

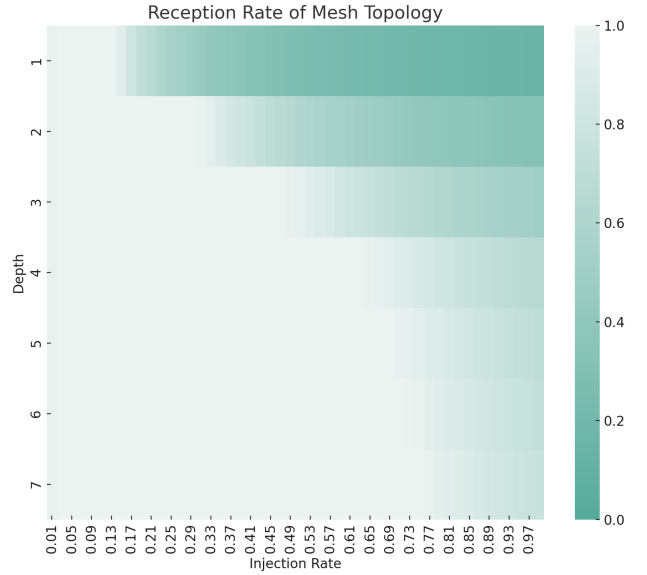


Fig. 8. Reception rate of mesh

Figure 8 and figure 9 show that mesh has higher throughput, since torus is more susceptible to deadlock.

B. Simple Bubble, Critical Bubble, Normal wormhole

In terms of latency, Normal wormhole performs best, followed by critical bubble, simple bubble is the worst. This is due to the utilization of buffer in channel. Simple uses the least amount, normal wormhole uses the most, critical bubble stands in between.

By comparing fig 9, 11 and 12, we can see bubble flow control prominently increases the throughput of network because it avoids deadlock. Critical bubble performs worse than simple bubble because when workload is high, only one bubble in network results in starvation.

Moreover, we run experiments on two other synthetic types, shuffle and transpose. Results are fig 13, 14, 15, 16, 17, 18. In transpose, simple bubble and critical bubble performs similarly. In shuffle, critical bubble has lower latency and higher throughput than simple bubble.

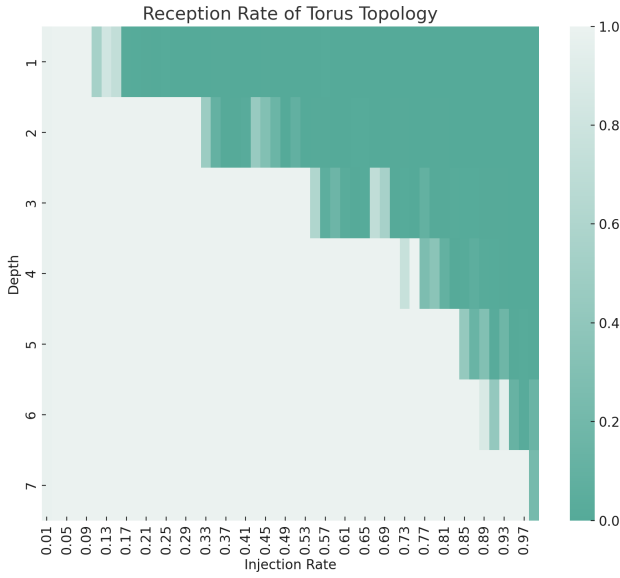


Fig. 9. Reception rate of torus

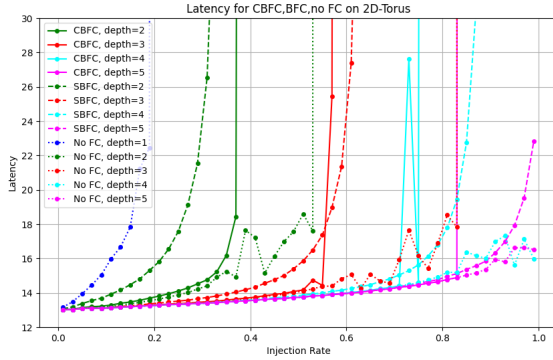


Fig. 10. Latency of three flow control

C. Multiple critical bubble

Fig 19 shows that increasing number of critical bubbles can reduce latency, since more bubbles means less congestion.

Fig 20, 21 and 12 show that more bubbles can increase throughput. Actually, multiple bubble is a trade-off scheme between simple bubble and no bubble, which addresses the issue of starvation in critical bubble.

VI. CONCLUSION

Normal wormhole scheme has the lowest latency due to full utilization of buffer, but it is not robust, because it can easily generate deadlock when load is high. Simple bubble scheme fixes this problem, hence has high throughput even when injection rate is close to 1, the limitations are higher latency and poor buffer utilization (can only work when buffer size ≥ 2). To get a trade off between the two schemes, critical bubble scheme is proposed. We can adjust the number of critical bubbles in the network to achieve a balance. An insight is that if we can adaptively adjust the number of bubbles in

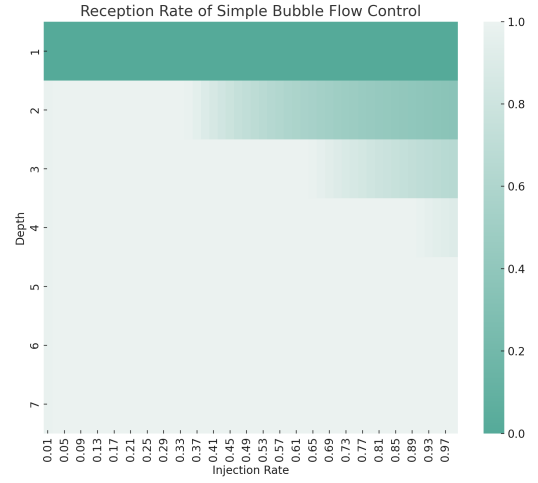


Fig. 11. Reception rate of Simple bubble

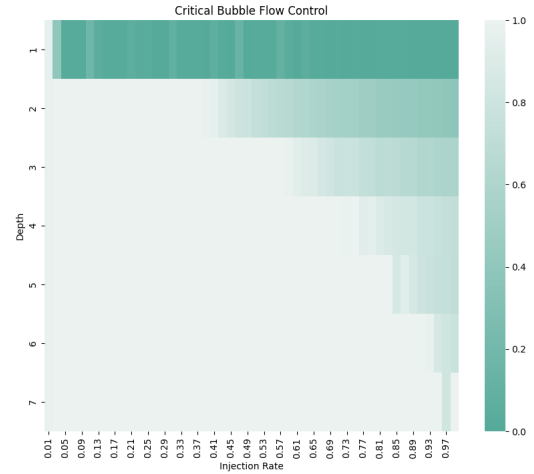


Fig. 12. Reception rate of critical bubble

network and apply proper routing algorithm, the performance may be better and the network will be more robust.

VII. DIVISION OF LABOR

- Student A: 2D-torus, Simple bubble scheme, Slides, 60% report
- Student B: Critical bubble scheme, Multiple critical bubble scheme, experiment tools, 40% report.

REFERENCES

- [1] L. Chen, R. Wang and T. M. Pinkston, "Critical Bubble Scheme: An Efficient Implementation of Globally Aware Network Flow Control," 2011 IEEE International Parallel & Distributed Processing Symposium, Anchorage, AK, USA, 2011, pp. 592-603, doi: 10.1109/IPDPS.2011.63.
- [2] L. Chen and T. M. Pinkston, "Worm-Bubble Flow Control," 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA), Shenzhen, China, 2013, pp. 366-377, doi: 10.1109/HPCA.2013.6522333.

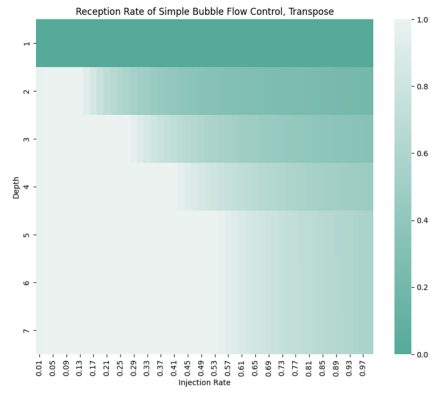


Fig. 13. Reception rate of simple bubble, transpose

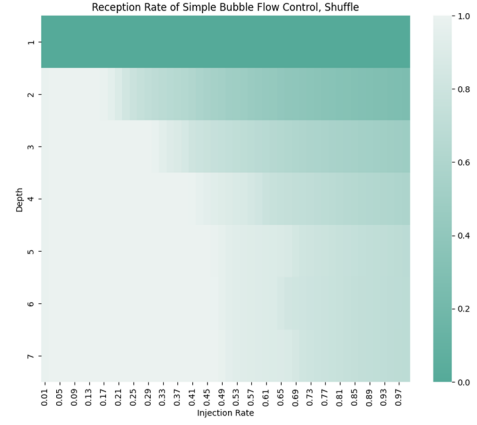


Fig. 16. Reception rate of simple bubble, shuffle

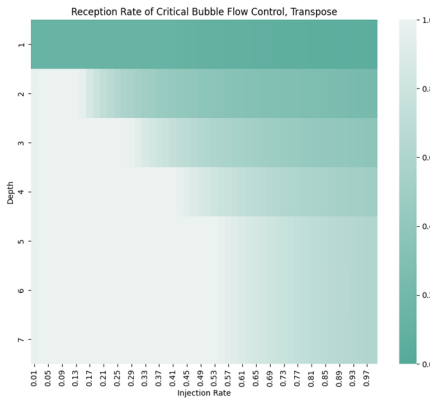


Fig. 14. Reception rate of critical bubble, transpose

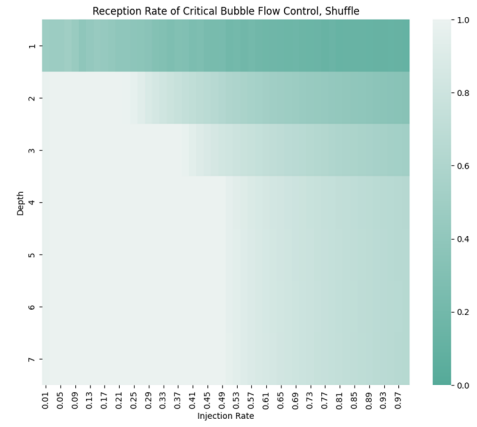


Fig. 17. Reception rate of critical bubble, shuffle

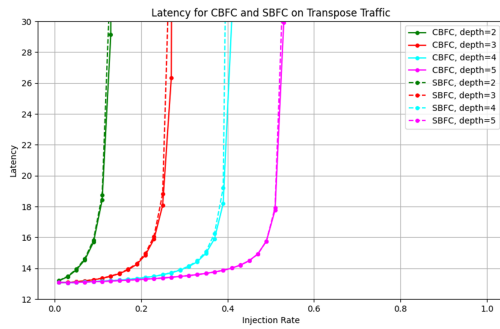


Fig. 15. Latency, transpose

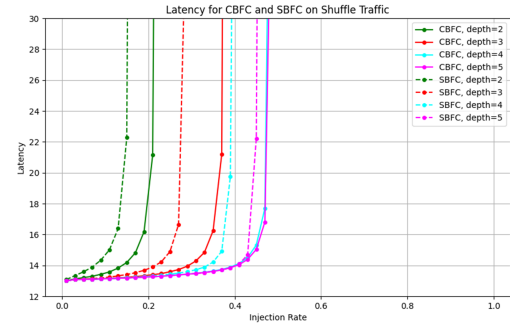


Fig. 18. Latency, shuffle

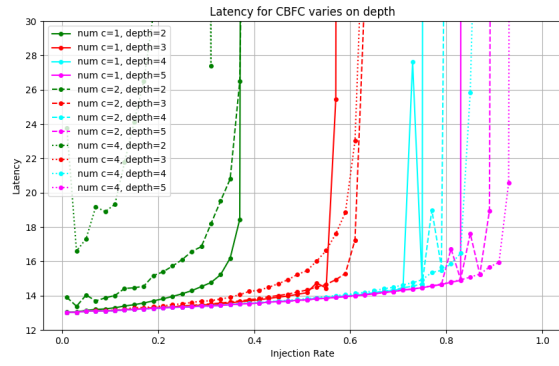


Fig. 19. Latency of multiple critical bubble

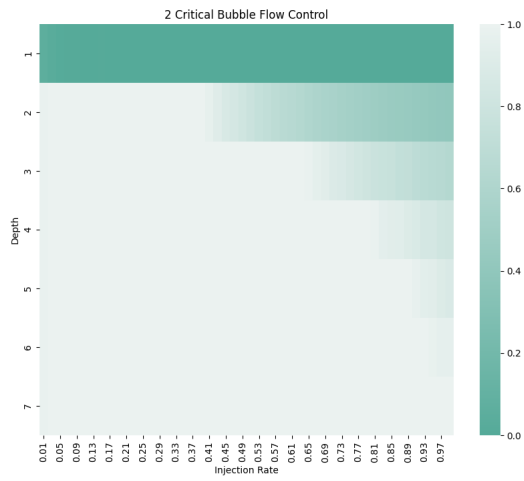


Fig. 20. Reception rate of 2 critical bubble

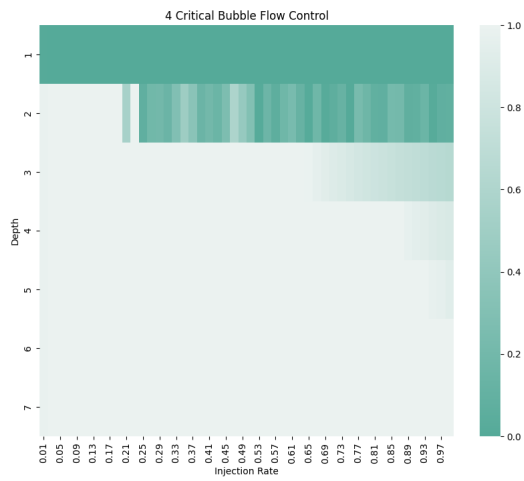


Fig. 21. Reception rate of 4 critical bubble